

```
rootReducer = combineReducers({ contacts });
```

The *rootReducer* will pass the action and the respective state to each reducer. We will implement the reducer function *contactReducer* accepting two parameters—state and action. The reducer is supposed to handle the action it is interested in and return the new state. The important thing to understand is that the reducer is a pure function, and it should not mutate the parameters and return a new state in an immutable way using only the values passed in the parameters. In *contactReducer*, we will handle the actions – successful loading of contacts, successful addition of a contact and successful deletion of a contact.

```
export default function contactReducer(state=initialState,
contacts, action) {
  switch (action.type) {
    case 'LOADED_CONTACTS':
      // Return contacts

    case 'ADDED_CONTACT':
      // Add new contact to state and return

    case 'DELETED_CONTACT':
      // Delete contact from state and return

    default:
      // Return state passed as parameter
  }
}
```

Implementing the actions

Create a file *actionTypes.js* in the folder *actions* to store all the action names as a constant. Having the action types as a constant will help in better maintenance of code rather than having them directly as a string.

Next, create a file *contactActions.js*, in which we will implement various actions related to Contacts. We will implement the actions—loading of contacts, adding a contact and deleting a contact. As we will communicate using the REST API calls with a server in the backend, we will make asynchronous calls and, hence, we will use the middleware, *redux-thunk*, to perform asynchronous dispatch. The *thunk* function will invoke the Contacts API (discussed below), which will communicate with the server and fetch or add/modify the data based on the action. The Contacts API will use the npm module *axios* to make REST API calls to the Express server. The Contacts API will perform the necessary task and return either success or error. If success is returned, then the corresponding success action—load, add or delete—will be dispatched, after which the reducer code discussed above will get executed.

```
export function loadContacts() {
```

```
  return function(dispatch) {
    // Invoke Contacts API to load contacts and process
    // successful result or error
  };
}

export function addContact(contact) {
  return function(dispatch) {
    // Invoke Contacts API to add contact and process
    // successful result or error
  };
}

export function deleteContact(id) {
  return function(dispatch) {
    // Invoke Contacts API to delete contact and process
    // successful result or error
  };
}
```

Implementing the API invocation

Create a file *contactApi.js* in the folder *api*. We will implement the APIs *getAllContacts*, *saveContact* and *deleteContact*. To implement the invocation of these APIs, we will issue REST API calls to the Express server using the npm module *axios*.

```
export default class ContactApi {
  static getAllContacts() {
    // Invoke REST endpoint '/api/contacts' using axios.get
  }

  static saveContact(contact) {
    // Invoke REST endpoint '/api/contacts' using axios.post
  }

  static deleteContact(id) {
    // Invoke REST endpoint '/api/contacts/<id>' using axios.delete
  }
}
```

Implementing the UI components

We will now use React to implement the UI components. Also, we will use the *react-redux* module to interface React with Redux.

Create a file *ContactsComponent.js* in the folder *components*. Here, we will implement the top-level React container component, which will interface with the Redux store. We will write the method *render*, which will return the JSX to generate the Virtual DOM for the Contacts display and also a form to get the inputs from the user for adding a contact. Also, each contact will render itself using a separate React component, *ContactComponent*. Then, we will write